JAVASCRIPT

Assignment Answers – Questions 1 to 10

Theory • Examples • DOM • Events • Async JavaScript • Student Management App

Subject: JavaScript Programming

Includes a complete mini project with HTML, CSS and JavaScript

1. What is JavaScript? Explain its role and differentiate it from HTML and CSS.

JavaScript is a high-level programming language mainly used to make web pages interactive and dynamic. It can respond to user actions, validate forms, update page content, communicate with servers and build web applications.

HTML defines the structure and content of a page; **CSS** controls its appearance and layout; **JavaScript** controls behaviour and interactivity.

Technology	Main role	Example
HTML	Structure and content	Heading, paragraph, form
CSS	Design and presentation	Colour, font, spacing
JavaScript	Behaviour and logic	Button click, validation, calculations

Example: HTML creates a button, CSS styles it, and JavaScript can display a message when the button is clicked.

2. Explain var, let and const with scope differences.

var: function-scoped (or globally scoped when declared outside a function); it can be redeclared and reassigned. **let:** block-scoped; it can be reassigned but not redeclared in the same scope. **const:** block-scoped; it cannot be reassigned after initialization. A const object or array can still have its contents changed.

```
var city = 'Salem';
var city = 'Chennai'; // redeclaration allowed

let age = 20;
age = 21;             // reassignment allowed

const country = 'India';
// country = 'Japan'; // TypeError: reassignment not allowed

if (true) {
    var a = 10;       // available outside this block
    let b = 20;       // block-scoped
    const c = 30;     // block-scoped
}
console.log(a);       // 10
// console.log(b);    // ReferenceError
```

Best practice: Prefer const by default; use let when a variable must be reassigned. Avoid var in modern JavaScript.

3. Explain JavaScript data types with examples.

JavaScript has primitive and non-primitive (reference) data types.

Primitive types: String, Number, Boolean, Undefined, Null, BigInt and Symbol. Primitive values are not objects.

```
let name = 'Anu';             // String
let age = 20;                 // Number
let isPassed = true;          // Boolean
let result;                   // Undefined
let selectedValue = null;     // Null
let largeNumber = 123456789n;  // BigInt
let uniqueId = Symbol('id');  // Symbol
```

Non-primitive types: Objects, arrays and functions. They are reference types.

```
let student = { name: 'Anu', age: 20 }; // Object
let marks = [80, 90, 75];               // Array
function greet() { return 'Hello'; }    // Function
```

Use `typeof` to inspect many JavaScript types. Note: `typeof null` returns `'object'` for historical reasons.

4. What are functions? Give normal, expression and arrow function examples.

A function is a reusable block of code that performs a task. It can accept parameters and return a value.

1. Normal function declaration:

```
function add(a, b) {
    return a + b;
}
```

```
console.log(add(5, 3)); // 8
```

2. Function expression:

```
const multiply = function(a, b) {
    return a * b;
};
console.log(multiply(4, 3)); // 12
```

3. Arrow function:

```
const subtract = (a, b) => {
    return a - b;
};
console.log(subtract(9, 4)); // 5

// Short form for one expression:
const square = n => n * n;
console.log(square(5)); // 25
```

5. Explain arrays and objects. Create an array of students and one student object.

An **array** stores an ordered list of values and uses zero-based indexes. An **object** stores related information as key-value pairs.

```
const students = [
    { id: 1, name: 'Arun', course: 'MERN', marks: 85 },
    { id: 2, name: 'Meena', course: 'Python', marks: 92 },
    { id: 3, name: 'Kavin', course: 'Java', marks: 68 }
];

const student = {
    id: 1,
    name: 'Arun',
    course: 'MERN',
    marks: 85
};

console.log(students[0].name); // Arun
console.log(student.course);   // MERN
```

6. Explain map(), filter(), find(), forEach() and reduce() with examples.

These array methods help process arrays. The examples use the same marks array:

```
const marks = [60, 75, 90, 45, 80];

// map(): creates a new array by transforming each item
const bonusMarks = marks.map(mark => mark + 5);
// [65, 80, 95, 50, 85]

// filter(): creates a new array with items that pass a condition
const passed = marks.filter(mark => mark >= 50);
// [60, 75, 90, 80]

// find(): returns the first item that passes a condition
const firstHighMark = marks.find(mark => mark > 80);
// 90

// forEach(): runs a function once for each item
marks.forEach(mark => console.log(mark));

// reduce(): combines items into one value
const total = marks.reduce((sum, mark) => sum + mark, 0);
// 350
const average = total / marks.length;
// 70
```

Summary: map transforms; filter selects matching items; find returns the first match; forEach performs an action for each item; reduce combines values into one result.

7. What is DOM manipulation? Change text and style of an HTML element.

The **Document Object Model (DOM)** represents an HTML page as a tree of objects. JavaScript can select elements and change their text, attributes, styles or structure.

```
<!DOCTYPE html>
<html>
<body>
```

```
    <h2 id="heading">Old Heading</h2>
    <button id="changeBtn">Change</button>

    <script>
        const heading = document.getElementById('heading');
        const button = document.getElementById('changeBtn');

        button.addEventListener('click', () => {
            heading.textContent = 'Welcome to JavaScript!';
            heading.style.color = 'blue';
            heading.style.fontSize = '30px';
        });
    </script>
</body>
</html>
```

When the button is clicked, JavaScript changes the heading text, colour and font size.

8. Explain JavaScript events and event listeners. Create a click event.

An **event** is an action or occurrence in the browser, such as a click, key press, form submission or page load. An **event listener** waits for a specified event and runs a callback function when it occurs.

```
<!DOCTYPE html>
<html>
<body>
    <button id="myButton">Click Me</button>
    <p id="message"></p>

    <script>
        const button = document.getElementById('myButton');

        button.addEventListener('click', () => {
            document.getElementById('message').textContent =
                'Button clicked successfully!';
        });
    </script>
</body>
</html>
```

Here, `addEventListener('click', ...)` runs the callback when the user clicks the button.

9. What are Promises and async/await?

Asynchronous JavaScript allows an operation to start without blocking the rest of the program. A **Promise** represents a future result: it can be pending, fulfilled or rejected. **async/await** provides a clearer syntax for working with Promises. An async function always returns a Promise.

Promise example:

```
const task = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve('Task completed!');
    }, 1000);
});

task.then(result => console.log(result))
    .catch(error => console.log(error));
```

async/await example:

```
function waitOneSecond() {
    return new Promise(resolve => {
        setTimeout(() => resolve('Data loaded!'), 1000);
    });
}

async function getData() {
    try {
        const result = await waitOneSecond();
        console.log(result);
    } catch (error) {
        console.log('Error:', error);
    }
}

getData();
```

`await` pauses that async function until the Promise settles; it does not block the browser's main thread.

10. Build a mini Student Management application using HTML, CSS and JavaScript.

The following is a complete single-file application. Create a file named **index.html**, paste all the code, save it, and open it in a browser. It adds students, displays a table, calculates the average marks and shows pass/fail status (pass mark: 50).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Management</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f2f5f9;
            margin: 0;
            padding: 25px;
        }
        .container {
            max-width: 850px;
            margin: auto;
            background: white;
            padding: 22px;
            border-radius: 10px;
            box-shadow: 0 2px 12px #00000018;
        }
        h1 { color: #17365d; text-align: center; }
        .form-row {
            display: flex;
            flex-wrap: wrap;
            gap: 10px;
            margin-bottom: 15px;
        }
        input {
            flex: 1;
            min-width: 130px;
            padding: 10px;
            border: 1px solid #bbb;
            border-radius: 5px;
        }
        button {
            padding: 10px 16px;
            background: #1769aa;
            color: white;
            border: 0;
            border-radius: 5px;
            cursor: pointer;
        }
        button:hover { background: #104d80; }
        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 18px;
        }
        th, td {
            border: 1px solid #d5dce5;
            padding: 10px;
            text-align: left;
        }
        th { background: #17365d; color: white; }
        .pass { color: green; font-weight: bold; }
        .fail { color: #c62828; font-weight: bold; }
        #average { font-weight: bold; margin-top: 18px; }
        #message { color: #c62828; }
    </style>
</head>
<body>
<div class="container">
    <h1>Student Management System</h1>

    <form id="studentForm">
        <div class="form-row">
            <input id="name" type="text" placeholder="Student name" required>
            <input id="roll" type="text" placeholder="Register number" required>
            <input id="marks" type="number" placeholder="Marks (0-100)"
                   min="0" max="100" step="0.01" required>
            <button type="submit">Add Student</button>
        </div>
    </form>

    <p id="message" role="status"></p>

    <table>
        <thead>
            <tr>
```

```
                <th>Register No.</th>
                <th>Name</th>
                <th>Marks</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody id="studentTable"></tbody>
    </table>

    <p id="average">Average Marks: 0.00</p>
</div>

<script>
    const students = [];

    const form = document.getElementById('studentForm');
    const tableBody = document.getElementById('studentTable');
    const averageText = document.getElementById('average');
    const message = document.getElementById('message');

    form.addEventListener('submit', function(event) {
        event.preventDefault();

        const name = document.getElementById('name').value.trim();
        const roll = document.getElementById('roll').value.trim();
        const marks = Number(document.getElementById('marks').value);

        if (!name || !roll || !Number.isFinite(marks) ||
            marks < 0 || marks > 100) {
            message.textContent = 'Enter valid student details and marks from 0 to 100.';
            return;
        }

        const duplicate = students.some(student => student.roll === roll);
        if (duplicate) {
            message.textContent = 'Register number already exists.';
            return;
        }

        students.push({ name, roll, marks });
        message.textContent = '';
        form.reset();
        displayStudents();
    });

    function displayStudents() {
        tableBody.innerHTML = '';

        students.forEach(student => {
            const row = document.createElement('tr');

            const values = [
                student.roll,
                student.name,
                student.marks.toFixed(2),
                student.marks >= 50 ? 'Pass' : 'Fail'
            ];

            values.forEach((value, index) => {
                const cell = document.createElement('td');
                cell.textContent = value;

                if (index === 3) {
                    cell.className = student.marks >= 50 ? 'pass' : 'fail';
                }

                row.appendChild(cell);
            });

            tableBody.appendChild(row);
        });

        const total = students.reduce(
            (sum, student) => sum + student.marks, 0
        );

        const average = students.length ? total / students.length : 0;
        averageText.textContent = 'Average Marks: ' + average.toFixed(2);
    }
</script>
</body>
</html>
```

How it works:

- The form collects the student's name, register number and marks.
- The submit event validates the values and prevents the page from reloading.
- Student records are stored in a JavaScript array and displayed in the table.

• `reduce()` calculates the total marks; dividing by the number of students gives the average.
• Marks of 50 or above are shown as Pass; marks below 50 are shown as Fail.

Note: This beginner project stores data only in memory. Refreshing the browser clears the student list. A database or localStorage can be added for persistent storage.

Result

Thus, the JavaScript concepts and practical programs have been demonstrated, including variables, data types, functions, arrays, objects, array methods, DOM manipulation, events, asynchronous programming and a mini Student Management application.